

Lecture 12: Clustering

Clustering opens the machine-learning unit: it is **unsupervised learning**, the problem of making unlabeled data organize itself into “natural clusters,” formalized as an optimization problem and attacked by two workhorse algorithms — hierarchical clustering and k-means.

The Machine Learning Paradigm: Observe, Infer, Predict

Every learning method in this course fits a three-step skeleton:

1. **Observe** a set of examples — the **training data**.
2. **Infer** something about the process that generated the data.
3. **Predict** on previously unseen examples — the **test data**.

Within that skeleton there are two flavors of learning:

- **Supervised learning:** we are given feature/label pairs — each example comes with the “correct answer” — and must find a rule that predicts the label of a previously unseen input. This will occupy most of the course.
- **Unsupervised learning:** we get feature vectors *only*, with no labels and no teacher. The job is to group them into **natural clusters** so the data organizes itself.

Clustering is the canonical unsupervised task. Wikipedia situates it as a main task of **exploratory data analysis**, used across pattern recognition, image analysis, information retrieval, bioinformatics, data compression, computer graphics, and machine learning. It has accumulated many near-synonyms — *automatic classification*, *numerical taxonomy*, *botryology* (from Greek *botrys*, “grape”), *typological analysis*, *community detection* — differing mainly in how results are used: in data mining the resulting groups are the point, while in automatic classification the discriminative power is what matters. The idea is old: it originated in anthropology with Driver and Kroeber (1932), entered psychology through Zubin (1938) and Tryon (1939), and was famously used by Cattell (1943) for personality-trait classification.

Formalizing “Natural Clusters”: Variability, Dissimilarity, and Constraints

The key insight of the lecture: **clustering is an optimization problem**. We need a quantity that says one grouping is better than another.

For a single cluster c , define its **variability** — how spread out its points are around its center:

$$\text{variability}(c) = \sum_{e \in c} (\text{distance}(\text{mean}(c), e))^2$$

This is a variance-like measure: squared distances from the cluster mean. For a whole set of clusters C , define the **dissimilarity** as the total spread:

$$\text{dissimilarity}(C) = \sum_{c \in C} \text{variability}(c)$$

Why sum rather than average? Because a big-and-bad cluster is worse than a small-and-bad one: a large cluster that is still spread out is a bigger sin than a tiny scattered one. We deliberately do *not* normalize by cluster size. (Keep this in mind — it turns out to be exactly the standard k-means objective, as shown below.)

Why isn’t minimizing dissimilarity the whole problem? Because the trivial solution wins: put every example in its own cluster. Each cluster has one point, its mean is that point, every distance is zero, and dissimilarity is zero — a perfect score that reveals nothing about the data’s structure. So we must impose a **constraint**; typical choices are a minimum distance between clusters, or fixing the number of clusters. Constraints are what keep the trivial solution at bay and make the problem interesting.

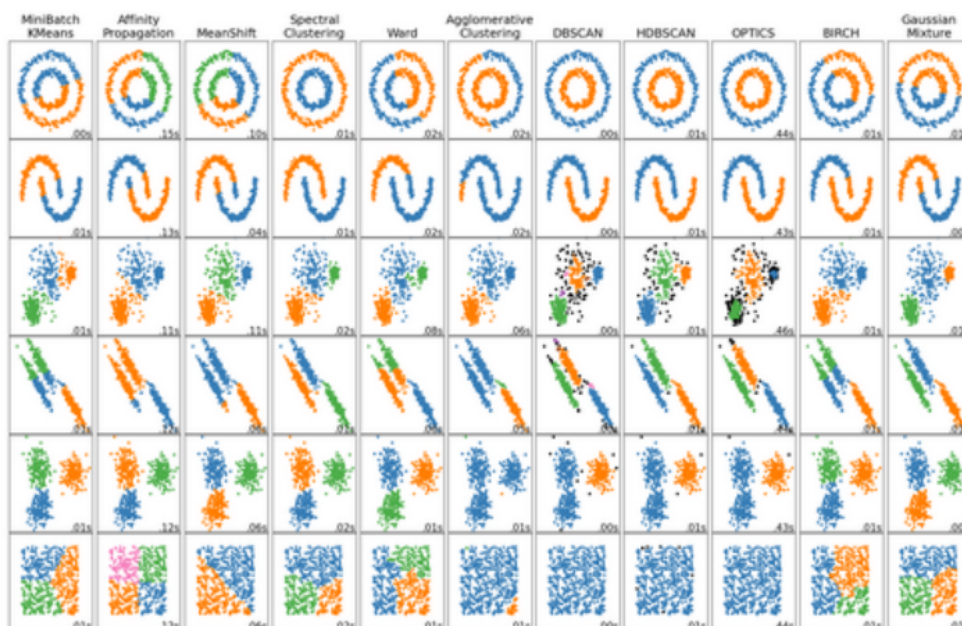
Wikipedia reinforces how slippery this formalization is: the notion of a “cluster” **cannot be precisely defined**, which is precisely why over 100 clustering algorithms exist. Common cluster models include groups with small distances between members, dense areas of the data space, intervals, or particular statistical distributions; a clustering may also specify relationships among clusters, such as a hierarchy embedded in other clusters. Consequently clustering is really a **multi-objective optimization** problem, and in practice it is not a one-shot automatic task but an **iterative process of knowledge discovery**: preprocessing and parameters (distance function, density threshold, expected number of clusters) get tuned until results have the desired properties.

There Is No Objectively Correct Clustering

A sobering theoretical result frames everything that follows. An axiomatic approach shows it is **impossible for any clustering method to satisfy three fundamental properties simultaneously**:

- **Scale invariance** — results unchanged under proportional scaling of distances;
- **Richness** — all possible partitions of the data are achievable;
- **Consistency** — agreement between distances and the clustering structure.

As Wikipedia puts it, “clustering is in the eye of the beholder”: there is no objectively correct algorithm, and the right choice usually must be made experimentally unless mathematics favors one cluster model. An algorithm built for one kind of model generally fails on data containing a radically different kind — for example, **k-means cannot find non-convex clusters**, and most traditional methods assume clusters are spherical, elliptical, or convex. The practical consequence: know what cluster model your algorithm implicitly assumes before trusting its output.



Source: Wikipedia, article “[Cluster analysis](#)”.

Hierarchical (Connectivity-Based) Clustering

The algorithm

Hierarchical clustering — Wikipedia calls it **connectivity-based clustering**, built on the idea that objects are more related to nearby objects than to distant ones — is beautifully simple:

1. Assign each item to its own cluster (N items $\rightarrow N$ singleton clusters).

2. Find the closest (most similar) pair of clusters and merge them.
3. Repeat until everything is one cluster of size N .

The cluster count walks down $N, N - 1, N - 2, \dots, 1$. Instead of producing a single partition, the algorithm builds a **hierarchy** of clusters that merge at different distances — a cluster can be understood by the maximum distance needed to connect its elements, and at different distance thresholds, different groupings appear.

The key question: what does “distance between clusters” mean?

Step 2 hides the hard part: when clusters contain many points, what is the distance between two *clusters*? Standard answers are the **linkage metrics**:

- **Single-linkage**: the *shortest* distance between any member of one cluster and any member of the other — clusters are as close as their nearest pair of points.
- **Complete-linkage**: the *greatest* such distance — clusters are only as close as their farthest-apart pair.
- **Average-linkage**: the average over all cross-cluster pairs (Wikipedia notes the average variants go by the names **UPGMA** and **WPGMA**).

Worked example: six American cities

Distances (miles): Boston–NY 206, Boston–Chicago 963, Boston–Denver 1949, Boston–SF 3095, Boston–Seattle 2979, NY–Chicago 802, NY–Denver 1771, NY–SF 2934, NY–Seattle 2815, Chicago–Denver 966, Chicago–SF 2142, Chicago–Seattle 2013, Denver–SF 1235, Denver–Seattle 1307, SF–Seattle 808.

Running the algorithm from singletons $\{\text{BOS}\}, \{\text{NY}\}, \{\text{CHI}\}, \{\text{DEN}\}, \{\text{SF}\}, \{\text{SEA}\}$:

1. Closest pair: Boston–NY at **206** $\rightarrow \{\text{BOS}, \text{NY}\}$.
2. Next: NY–Chicago at **802** $\rightarrow \{\text{BOS}, \text{NY}, \text{CHI}\}$.
3. Next: SF–Seattle at **808** $\rightarrow \{\text{SF}, \text{SEA}\}$.

Now the linkage metric decides the outcome. Denver sits between the two remaining clusters:

- **Single linkage**: $d(\text{DEN}, \{\text{BOS}, \text{NY}, \text{CHI}\}) = \min(1949, 1771, 966) = 966$, versus $d(\text{DEN}, \{\text{SF}, \text{SEA}\}) = \min(1235, 1307) = 1235$. Denver joins the **eastern** cluster: $\{\text{BOS}, \text{NY}, \text{CHI}, \text{DEN}\}, \{\text{SF}, \text{SEA}\}$.
- **Complete linkage**: $d(\text{DEN}, \{\text{BOS}, \text{NY}, \text{CHI}\}) = \max(1949, 1771, 966) = 1949$, versus $d(\text{DEN}, \{\text{SF}, \text{SEA}\}) = \max(1235, 1307) = 1307$. Denver joins the **western** cluster: $\{\text{BOS}, \text{NY}, \text{CHI}\}, \{\text{DEN}, \text{SF}, \text{SEA}\}$.

Same data, same algorithm — a different notion of “distance between clusters” flips the answer. (Average linkage gives $(1949 + 1771 + 966)/3 = 1562$ versus $(1235 + 1307)/2 = 1271$, siding with complete linkage here.) You must think about which linkage suits your problem.

Reading the output: the dendrogram

A major advantage of hierarchical clustering is that you can **select the number of clusters after the fact** using a **dendrogram** — a tree diagram (Greek “tree” + “drawing”) whose leaves are the individual items and whose internal nodes are merges. In a dendrogram the **y-axis shows the distance at which clusters merge** and the x-axis arranges objects so clusters appear as continuous branches; heights are **monotone increasing** with merger level, each node’s height proportional to the inter-group dissimilarity between its two daughters, while leaves sit at zero height. Cutting the tree at different heights yields different numbers of clusters — one run of the algorithm gives you every granularity for free.

Properties and cost

Hierarchical clustering is **deterministic** (same input, same output every time) and **flexible** (single, complete, or average linkage plug in freely). Its weakness is speed: the naïve algorithm is order n^3 , though $O(n^2)$ algorithms exist for some linkage criteria.

k-Means Clustering

Motivation and pseudocode

Where hierarchical clustering is slow, **k-means** is a fast, greedy algorithm best suited to when you *know* how many clusters you want — that is the k in the name:

1. Randomly choose k examples as the initial centroids.
2. Loop:
 - Create k clusters by assigning each example to its **closest centroid**.
 - Compute k new centroids by **averaging** the examples in each cluster.
 - If the centroids don't change, break — we've converged.

Wikipedia identifies this as the **iterative refinement** technique, known as **Lloyd's algorithm** (or “naïve k-means,” since faster alternatives exist). History: the term “k-means” is due to James MacQueen (1967), the idea goes back to Hugo Steinhaus (1956), and the standard algorithm was proposed by Stuart Lloyd at Bell Labs in 1957 for **pulse-code modulation** — representing analog signals with a limited set of discrete values, i.e., vector quantization — though it wasn't published until 1982; Edward Forgy published essentially the same method in 1965, hence “**Lloyd-Forgy algorithm**.” Early uses were signal processing and data compression; later, pattern recognition and large-scale data analysis.

Complexity of one iteration: each of the n points is compared against each of the k centroids, and each comparison costs whatever one distance computation costs — so $\$k \cdot n$

The Cluster class: examples plus a centroid

A **Cluster** bundles two things: a set of examples and a **centroid** — the point that represents the cluster's center. The constructor `__init__(self, examples)` *assumes* `examples` is a non-empty list of **Example** objects: we never build a cluster around nothing, because a centroid is defined as an average of members, and an average of zero points is undefined.

Three methods do the work:

- **update(self, examples)** replaces the cluster's membership with a new list of examples and returns *how far the centroid moved*. This returned displacement is exactly the quantity k-means monitors: when reassigning points no longer moves any centroid, we have converged.
- **computeCentroid()** computes the mean literally. It starts from a vector of zeros matching the examples' dimensionality, accumulates every member's feature vector, and divides by the member count:

$$\text{centroid} = \frac{1}{|S|} \sum_{x \in S} x$$

The result is wrapped as `Example('centroid', vals/len(self.examples))`. This reuse is deliberate: the centroid is itself an **Example**, so it can be treated exactly like any other point in the space — distances to it are computed with the same machinery as distances between data points. - **members()** is a generator that yields the cluster's examples one at a time, letting client code iterate over membership cleanly.

This matches the formal picture from clustering theory: k-means partitions n observations into k sets $S = \{S_1, \dots, S_k\}$ in which each observation belongs to the cluster with the **nearest mean**, where the mean (centroid) of cluster S_i is $\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$. Geometrically, assigning every point to its nearest center partitions the data space into **Voronoi cells** around the centroids.

Variability and dissimilarity: scoring a clustering

Two small functions turn the **Cluster** class into a quality metric:

- `variability()` sums, over all members, the *squared* distance to the centroid: `totDist += (e.distance(self.centroid))**2`. Squaring is the usual choice because it penalizes far-away points disproportionately — one badly placed point costs more than several mildly misplaced ones.
- `dissimilarity(clusters)` simply returns the sum of the variabilities of all clusters in a list. The smaller this total, the tighter — the better — the clustering. This function becomes the judge that ranks competing clusterings later.

This is precisely the textbook **within-cluster sum of squares (WCSS)** objective that k-means minimizes:

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2 = \arg \min_{\mathbf{S}} \sum_{i=1}^k |S_i| \text{Var } S_i$$

Minimizing WCSS is equivalent to minimizing the pairwise squared deviations of points within the same cluster, via the identity

$$|S_i| \sum_{x \in S_i} \|x - \mu_i\|^2 = \frac{1}{2} \sum_{x, y \in S_i} \|x - y\|^2,$$

and — since total variance is fixed — equivalent to *maximizing* the between-cluster sum of squares (BCSS), a deterministic relationship tied to the law of total variance. Note the subtlety: k-means minimizes within-cluster variances (squared Euclidean distances), **not** plain Euclidean distances. Minimizing ordinary Euclidean distance would be the harder Weber problem, whose optimum is the geometric median rather than the mean; variants such as k-medians and k-medoids target that criterion instead.

Z-scaling: putting attributes on the same footing

Before clustering patients, the lecture builds a thin domain wrapper: `class Patient(cluster.Example): pass`. A patient *is* an example — inheritance supplies all behavior, and no new code is needed.

But raw attributes must first be made comparable, and that is the job of `scaleAttrs(vals)`: convert to an array, compute `mean = sum(vals)/len(vals)` and `sd = numpy.std(vals)`, then return `(vals - mean)/sd`. This is **Z-scaling**, producing the **standard score**:

$$z = \frac{x - \mu}{\sigma}$$

The quiz answers fall out immediately: subtracting the mean shifts the transformed values to mean 0, and dividing by the standard deviation rescales them to standard deviation 1. Every attribute ends up on the same footing — so heart rate expressed in beats per minute cannot dominate another measurement merely because of its units. The z-score is inherently meaningful: it counts how many standard deviations a value sits above (positive) or below (negative) the mean, and because numerator and denominator carry the same units, the units cancel and z is **dimensionless**.

Why this matters for clustering specifically: as the statistics literature puts it, “*When the variables in a multivariate data set are on different scales, it makes more sense to calculate the distances after some form of standardization*” — and distance is everything in k-means. The same standardization is standard practice in principal components analysis, where variables with widely differing ranges are routinely standardized first. A classic comparison shows the payoff: a student scoring 1800 on the SAT ($\mu = 1500$, $\sigma = 300$) has $z = (1800 - 1500)/300 = 1$, while a student scoring 24 on the ACT ($\mu = 21$, $\sigma = 5$) has $z = (24 - 21)/5 = 0.6$ — so the SAT student performed better *relative to peers*, even though the raw numbers live on incomparable scales. One caveat worth knowing: computing a true z-score requires the population mean and standard deviation; estimating them from a sample yields the analogous t-statistic instead.

The data loader exposes this as a switch — `getData(toScale = False)` applies `scaleAttrs` to each attribute list (e.g., `hrList`) only when `toScale` is true — so experiments can be run with and without scaling to see why the choice matters.

The k-means algorithm: assignment and update to convergence

The driver is `kmeans(examples, k, verbose = False)`, following the classic recipe:

1. Get k **randomly chosen initial centroids** and create one cluster per centroid.
2. Iterate until the centroids no longer change:
 - Associate each example with its **closest centroid**.
 - Guard against degeneracy: `if len(c) == 0: raise ValueError('Empty Cluster')`. An empty cluster has no computable centroid — remember, `computeCentroid` divides by the member count — so failing loudly with a `ValueError` beats crashing mysteriously deep inside the arithmetic.
 - Call `update` on each cluster, which recomputes centroids and reports how far they moved; when nothing moves, we’ve converged.

This is **Lloyd’s algorithm** (also called “naïve k-means”), an *iterative refinement* technique alternating two steps: assignment of each observation to its nearest mean, and recomputation of the means. Convergence is guaranteed in a specific sense: the objective (WCSS) monotonically decreases after each iteration, yielding a nonnegative, monotonically decreasing sequence. The full optimization problem is **NP-hard**, but these efficient heuristics converge quickly to a *local* optimum — which is exactly why the starting point matters.

The method has deep roots: the idea goes back to Hugo Steinhaus (1956); Stuart Lloyd proposed the standard algorithm at Bell Labs in 1957 as a technique for **pulse-code modulation** — clustering signals to reduce data while preserving quality (vector quantization) — though it was only published in 1982; Edward Forgy published essentially the same method in 1965, hence “Lloyd–Forgy”; and James MacQueen coined the term “k-means” in 1967. Early applications were in signal processing and data compression, later spreading to pattern recognition and statistical classification.

Two relationships sharpen the picture. First, k-means is structurally similar to the **expectation–maximization algorithm for Gaussian mixtures**: both iteratively refine a model built from cluster centers, but k-means tends to find clusters of *comparable spatial extent*, whereas a Gaussian mixture model permits clusters of different shapes. Second, despite the similar name, k-means is *unsupervised* and only loosely related to the supervised **k-nearest neighbor** classifier; however, applying the 1-nearest-neighbor rule to the learned cluster centers yields the **nearest centroid classifier** (Rocchio algorithm), which classifies new data into the existing clusters.

Known limitations motivated the extensions: sensitivity to initial centroid placement and difficulty with non-spherical clusters led to methods like **fuzzy c-means** (points belong to multiple clusters with varying degrees of membership) and **kernel k-means** (kernel functions to identify non-linearly separable clusters).

trykmeans: many random restarts, keep the best

Because step 1 is random, different runs of k-means can land in *different* final clusterings — some good, some bad. The wrapper `trykmeans(examples, numClusters, numTrials, verbose=False)` addresses this head-on: it calls `kmeans` `numTrials` times and **returns the result with the lowest dissimilarity**. Each trial’s answer is scored by the `dissimilarity` metric from earlier (total summed variability), and the tightest clustering wins.

This closes the loop on the whole design: random starts feed an iterate-to-convergence loop guarded against empty clusters, repeated restarts hedge against unlucky initializations, and a single scalar quality measure picks the winner. The need for this hedging was recognized early in the algorithm’s history — sensitivity to initial centroid placement is one of the original, well-documented weaknesses of k-means, and it is precisely what improved initialization techniques and multi-restart schemes were developed to counteract.